Hausa News Topic Classification: Comparing Classical, Recurrent and Transformer Models

MAHE Digne · ML Techniques I · October 2026

Project links

- **GitHub repository:** Link
- **Demo video:** Link
- **Live system:** Link
- **Fine-tuned model:** Link

1. Abstract

Hausa is one of Africa's most widely spoken languages, yet it still has far fewer NLP tools than English or French. This project builds a system that reads a Hausa news article and assigns it one of seven topics: business, entertainment, health, politics, religion, sports or technology. I used the Hausa part of the MasakhaNEWS dataset (3,173 articles) and compared 18 experiments in three families: sparse TF-IDF models, BiLSTM/BiGRU networks with Word2Vec embeddings, and fine-tuned Transformers. The baseline (TF-IDF + Logistic Regression) reached a test macro-F1 of 0.871. The best model by validation score, a fully fine-tuned AfroXLMR-base with 512 tokens of input, reached 0.908 macro-F1 and 91.4% accuracy, a gain that is statistically significant (exact McNemar test, p = 0.017). Africa-focused pretraining mattered (+3.6 F1 over generic XLM-R) and full fine-tuning was essential, while sequence length and learning rate made little difference. Simple models were surprisingly competitive: a tuned TF-IDF model scored 0.885 and a BiLSTM with Word2Vec 0.888. Most remaining errors come from articles that genuinely sit between two topics, such as Hajj finances (business vs religion). The final model is deployed as a Gradio web app.

2. Introduction

News is one of the main ways people follow what is happening around them, and automatic topic labelling is useful for organising archives, recommending articles and monitoring media. African languages are still under-represented in NLP research, partly because labelled datasets are scarce (Joshi et al., 2020; Nekoto et al., 2020). Hausa is spoken by tens of millions of people across West Africa and has a healthy supply of online news, which makes it a realistic language for a useful classifier. It also has its own challenges: hooked letters (ɓ, ɗ, ƙ, ƴ) that are often typed inconsistently, a lot of borrowed English and Arabic vocabulary, and many Nigerian names and places.

Objective. Build, evaluate and deploy a Hausa news topic classifier, and understand *why* different model types behave the way they do. The questions I wanted to answer were:

1. How much better is a fine-tuned Transformer than a strong classical baseline?
2. Does Africa-focused pretraining help compared with a generic multilingual model?
3. Do Word2Vec embeddings, attention, or longer inputs help?
4. Where does the best model still fail, and why?

Target users. Newsrooms, researchers and developers working with Hausa text who need a quick way to tag or sort articles.

Note on language choice. The "no existing machine-translation system" requirement applies to translation projects. This is a classification project, so it does not apply here; for completeness, Hausa is in fact covered by existing MT systems such as NLLB-200 (NLLB Team, 2022).

3. Related Work

Datasets. MasakhaNEWS is the largest news topic classification dataset for African languages, covering 16 languages including Hausa. Its authors released baselines built from classical machine learning and fine-tuned language models (Adelani et al., 2023). Earlier work on Hausa and Yorùbá news headlines showed that multilingual Transformers and transfer learning help in low-resource African settings (Hedderich et al., 2020).

Models. TF-IDF features (Salton & Buckley, 1988) with linear classifiers are the standard strong baseline for topic classification (Pedregosa et al., 2011). Recurrent networks such as LSTMs (Hochreiter & Schmidhuber, 1997) and GRUs (Cho et al., 2014), optionally with attention (Bahdanau et al., 2015) and Word2Vec embeddings (Mikolov et al., 2013), were the dominant neural approach before Transformers (Vaswani et al., 2017; Devlin et al., 2019). Multilingual XLM-R (Conneau et al., 2020) covers Hausa, but it was trained mostly on high-resource languages. Two Africa-focused alternatives exist: AfroXLMR, which adapts XLM-R to 17 African languages plus a few widely spoken high-resource ones (Alabi et al., 2022), and AfriBERTa, a smaller model trained from scratch on 11 African languages, Hausa included (Ogueji et al., 2021).

Gap and contribution. I do not claim a new method. What this project adds is a controlled, single-pipeline comparison on Hausa: all models use the same splits and the same selection rule, and I run ablations the original baselines do not focus on (headline vs body, sequence length, frozen vs fine-tuned encoder), plus a statistical test and a detailed error analysis.

4. Dataset and Data Preparation

Source. I used the Hausa portion of MasakhaNEWS (Adelani et al., 2023), built from news websites, distributed through the Masakhane organisation on Hugging Face under the AFL-3.0 licence. It is real, naturally occurring text, which suits the aim of working with the genuine messiness of African-language data. No synthetic data was used.

Size and splits. I used the provided train/dev/test files (roughly 70/10/20):

Split	Articles	business	entertainment	health	politics	religion	sports	technology
Train	2,219	279	350	345	350	345	347	203
Dev	317	40	50	49	50	49	50	29
Test	637	80	100	99	100	99	100	59

Each article has a headline, body text, URL and a topic label. The seven categories were defined by the dataset authors, not by me.

What I found when exploring the data

- **Mild imbalance.** Technology is the smallest class (9.1%), business next (12.6%); the largest/smallest ratio is 1.72. Class proportions are almost identical across splits (Figure 1). Because of this I chose **macro-F1** as the main metric, since accuracy can hide weak small classes, and I used class-weighted loss/classifiers.
- **Long, uneven articles.** Mean length is 437 words (median 334, maximum 2,940). Entertainment articles are much shorter (median around 120 words), sports articles are very consistent in length (Figure 2). 57 training articles have fewer than 20 words, often just a photo caption or a show title.
- **Vocabulary.** The training set has about 1.0 million tokens and 30,402 distinct words, of which 45.5% appear only once. On unseen text, 2.0% (dev) and 2.4% (test) of tokens are out of vocabulary, so coverage is decent but there is a long tail of rare words and names.
- **Leakage check.** No duplicate rows in train, no URL overlap between any splits, and a single identical body text shared by train and test. That is negligible, but I report it for transparency.

Preprocessing. The model input is the headline and body joined as "headline. body". Missing values were replaced by empty strings. For the sparse and recurrent models, text was lowercased and tokenised with a regular expression that keeps Hausa hooked letters and apostrophes inside words; no stemming or stop-word removal was applied. Transformers use their own subword tokenisers. Long articles are truncated: 62% of training articles exceed 256 word tokens, and the share truncated by the Transformer tokenisers is 89% at 128, 76% at 256 and 52% at 512 subword tokens.

Limitations of the data. Only one source style (professional news), so informal text such as social media is untested; some articles are genuinely multi-topic and the labels are single-topic; and the religion class vocabulary is dominated by Islamic terms (see Section 7), which may not represent all religious content.

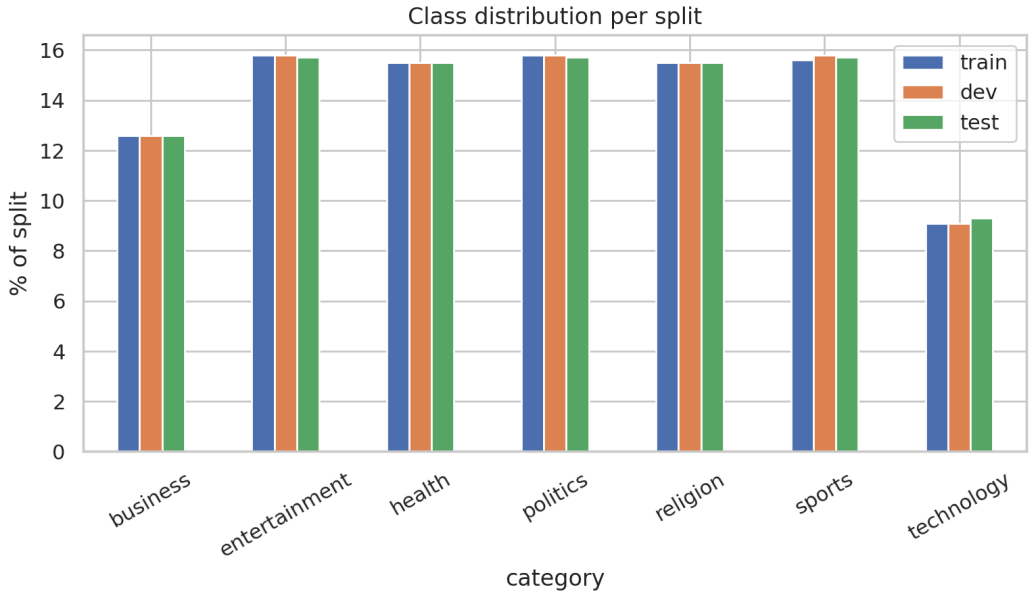


Figure 1. Class distribution per split. Proportions are almost identical in train, dev and test; technology is the smallest class.

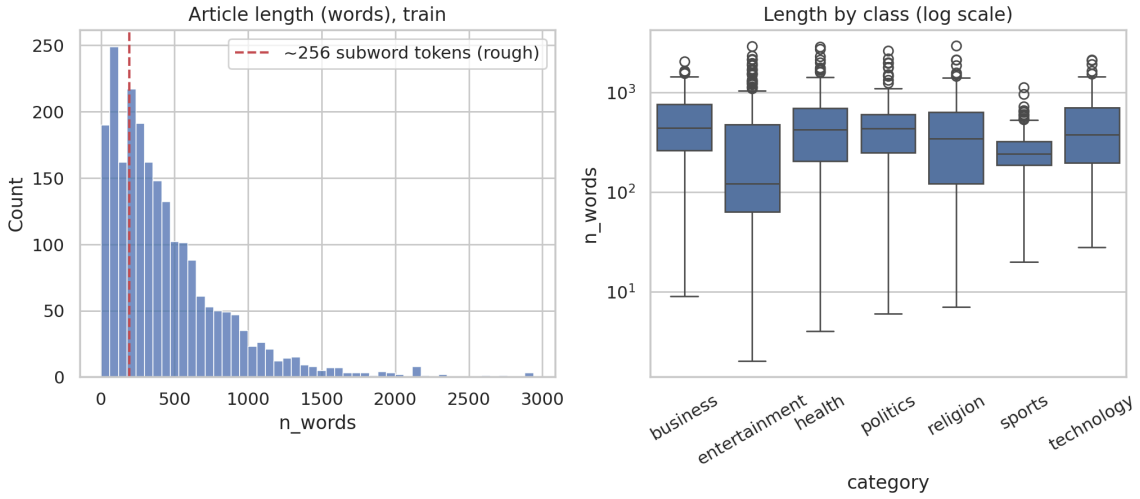


Figure 2. Article length (words) in the training set, overall and by class. Entertainment articles are shortest; sports articles are the most consistent.

5. Methodology

Fair-evaluation rule. Every model is trained on the training set and selected using **dev macro-F1** only. The test set is used once to report final numbers and never for tuning. Seed 42 was used everywhere.

Baseline (S1). TF-IDF on word unigrams and bigrams (sublinear term frequency, minimum document frequency 2) feeding a Logistic Regression with C = 1 and balanced class weights. I chose it because it is fast, hard to beat on topic tasks, and gives the neural models a clear number to beat.

Other sparse models (S2–S7). Stronger regularisation (C = 10), LinearSVC, Multinomial Naive Bayes, character n-grams (2–5), and headline-only and body-only inputs.

Recurrent models (N1–N4). Tokens → embedding layer (100-d) → bidirectional LSTM or GRU (128 hidden units per direction) → pooling over time steps (masked max-pooling or additive attention) → dropout 0.4 → linear layer → softmax. Articles are cut or padded to 256 tokens. The vocabulary (16,571 words with frequency ≥ 2) comes from the training set only. Word2Vec embeddings (skip-gram, window 5, 30 epochs) were also trained on **training text only**, so no dev/test text leaks in. Training used Adam (lr 1e-3), batch size 32, gradient clipping at 1.0, class-weighted cross-entropy, and early stopping on dev macro-F1 (patience 5, at most 25 epochs).

Transformers (T1–T7). A pretrained encoder plus a classification head on the first-token representation, fine-tuned end to end. I wrote the training loop by hand rather than using the `Trainer` class, so each step is explicit: AdamW (Loshchilov & Hutter, 2019) with weight decay 0.01, 10% linear warm-up then linear decay, fp16 mixed precision, gradient clipping, class-weighted cross-entropy, 5 epochs, and the best epoch chosen by dev macro-F1. Models: `Davlan/afro-xlmr-base` (AfroXLMR), `castorini/afriberta_base` (AfriBERTa) and `xlm-roberta-base` (generic XLM-R). Batch size was 16, except 8 for the 512-token run.

Existing resources vs my own work

Existing resources I used	What I implemented myself
MasakhaNEWS dataset; pretrained AfroXLMR, AfriBERTa, XLM-R; scikit-learn, PyTorch, Hugging Face Transformers, gensim, Gradio	Data exploration and leakage checks; all 18 experiment configurations; the RNN architectures with masked max/attention pooling; the hand-written Transformer fine-tuning loop; evaluation, McNemar test and error analysis; the Gradio app

Environment. Google Colab, Tesla T4 GPU, Python 3.13.15, PyTorch 2.11.0 (Paszke et al., 2019), Transformers 5.18.0 (Wolf et al., 2020), scikit-learn 1.6.1, gensim (Řehůřek & Sojka, 2010), Gradio (Abid et al., 2019).

6. Experiments and Results

The table lists every experiment. Selection used dev macro-F1; the test columns are for reporting only. Each configuration was run once with seed 42.

ID	Approach	Dev macro-F1	Test accuracy	Test macro-F1	Train time (s)
S1	**TF-IDF word (1–2) + LogReg C=1 (baseline)**	0.889	0.881	**0.871**	14
S2	TF-IDF + LogReg C=10	0.909	0.893	0.885	17
S3	TF-IDF + LinearSVC	0.902	0.895	0.884	3
S4	TF-IDF + Multinomial NB	0.861	0.856	0.828	3
S5	TF-IDF char n-grams (2–5) + LogReg C=10	0.894	0.892	0.883	28
S6	Headline only, TF-IDF + LogReg	0.816	0.799	0.790	1
S7	Body only, TF-IDF + LogReg	0.894	0.882	0.872	12
N1	BiLSTM, random embeddings, max-pool	0.837	0.810	0.800	49
N2	BiLSTM + Word2Vec, max-pool	0.905	0.896	0.888	32
N3	BiGRU + Word2Vec, max-pool	0.895	0.885	0.871	41
N4	BiGRU + Word2Vec, attention pooling	0.906	0.896	0.884	20
T1	XLM-R-base (generic), len 256, lr 2e-5	0.876	0.876	0.867	287
T2	AfriBERTa-base, len 256, lr 5e-5	0.904	0.915	0.910	161
T3	AfroXLMR-base, len 128, lr 2e-5	0.894	0.912	0.905	160

ID	Approach	Dev macro-F1	Test accuracy	Test macro-F1	Train time (s)
T4	AfroXLMR-base, len 256, lr 2e-5	0.901	0.911	0.903	232
T5	**AfroXLMR-base, len 512, lr 2e-5, bs 8 (selected)**	**0.916**	**0.914**	**0.908**	458
T6	AfroXLMR-base, len 256, lr 5e-5	0.916	0.912	0.906	220
T7	AfroXLMR-base, frozen encoder, len 256, lr 1e-3	0.839	0.821	0.808	94

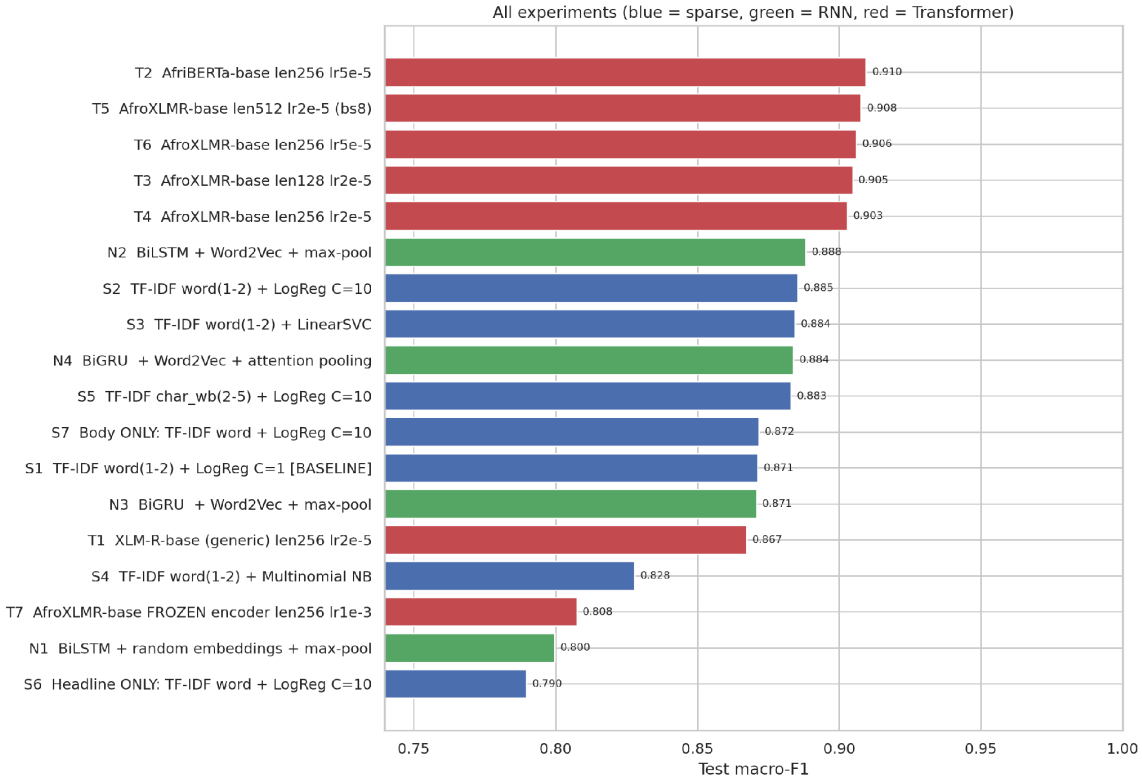


Figure 3. Test macro-F1 for all 18 experiments (blue = sparse, green = RNN, red = Transformer).

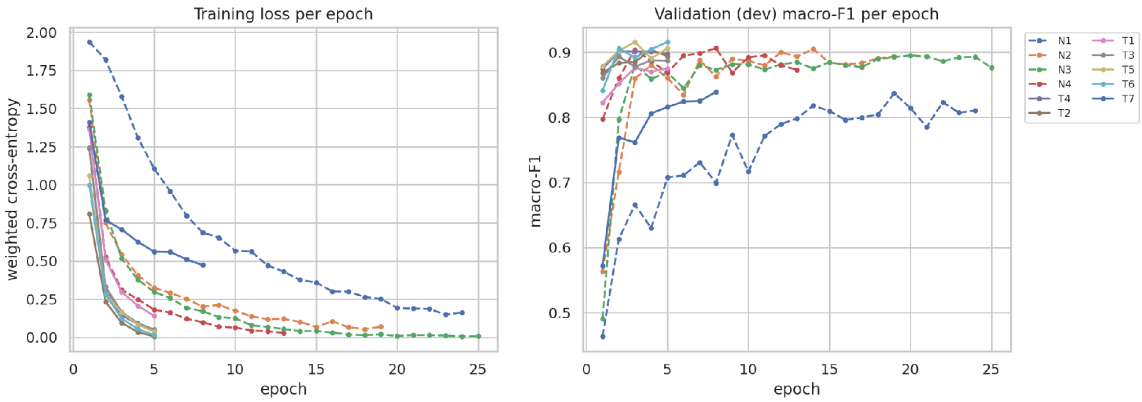


Figure 4. Training loss and dev macro-F1 per epoch for the neural models (dashed = RNN N1–N4, solid = Transformer T1–T7).

What the experiments taught me

- **The Transformer beats the baseline, and not by luck.** T5 improved macro-F1 from 0.871 to 0.908 and accuracy from 88.1% to 91.4%. Out of the articles where the two models disagreed, T5 was right on 46 and the baseline on 25 (exact McNemar test, McNemar, 1947; p = 0.017). A fairer yardstick is the *tuned* sparse model (S2, 0.885), where the gap shrinks to about 2.3 points; I did not run a significance test for that comparison.
- **Tuning the baseline was cheap and worthwhile.** Moving from C = 1 to C = 10 gained 1.4 points (S1 → S2). The baseline was deliberately the untuned default, so some of the "Transformer gain" is really "tuning gain".
- **Classifier choice.** LogReg and LinearSVC were equivalent (0.885 vs 0.884). Naive Bayes was clearly worse (0.828), likely because it handles correlated word and bigram features and class imbalance poorly. Character n-grams did not beat word n-grams (0.883), so spelling variation was not a big enough problem to need them.
- **Which part of the article matters.** The body alone (0.872) is far stronger than the headline alone (0.790), but combining both is best (0.885), so headlines add a little extra signal.
- **Word2Vec helped a lot for RNNs.** Starting from random embeddings (N1, 0.800) versus Word2Vec (N2, 0.888) is an 8.8-point jump. With only 2,219 training articles, the network cannot learn good word meanings from scratch. N1 also trained slowly and its dev score fluctuated, a sign of noisy learning (Figure 4).
- **LSTM vs GRU, and attention.** The BiLSTM (0.888) beat the BiGRU (0.871) with max-pooling. Attention pooling gave the GRU +1.3 points (0.884) and reached its best dev score in 8 epochs instead of 20, so it learned faster. These differences are small and come from single runs, so I treat them as suggestive only.
- **More complex is not automatically better.** Neither RNN beat a well-tuned TF-IDF model by a meaningful margin. For topic classification, the topic is mostly signalled by keywords (see Section 7), which bag-of-words models capture well, and the RNNs see only the first 256 tokens.
- **Pretraining matters.** With identical settings, AfroXLMR scored 0.903 against 0.867 for generic XLM-R (T4 vs T1): +3.6 points from pretraining on African languages. AfriBERTa (0.910) was as good as AfroXLMR despite being smaller and faster to train, though it also used a different learning rate, so that comparison is not perfectly controlled.
- **Sequence length and learning rate barely matter.** Lengths of 128, 256 and 512 gave 0.905, 0.903 and 0.908, and learning rates of 2e-5 and 5e-5 gave 0.903 and 0.906. The 512-token run took twice as long (458 s vs 232 s) for little benefit. This suggests the first 128 tokens (headline plus opening) carry most of the topic information.
- **Full fine-tuning is necessary.** Freezing the encoder and training only the head dropped the score to 0.808.
- **Training behaviour.** The fine-tuned Transformers reached their best dev score within 2–5 epochs while training loss fell to nearly zero, so more epochs would mostly cause overfitting (Figure 4).
- **How T5 was chosen.** T5 had the highest dev macro-F1 (0.9159, essentially tied with T6 at 0.9158), so it was selected by the pre-set rule. Its test score (0.908) is not the highest overall; T2 reached 0.910. With 637 test articles, one article is about 0.16 accuracy points, and all

fine-tuned African-language models lie within about 0.7 points of each other, so I treat them as roughly equal.

Per-class results (baseline S1 vs T5, test F1)

Class	S1	T5
business	0.759	0.844
entertainment	0.891	0.891
health	0.911	0.950
politics	0.890	0.905
religion	0.908	0.934
sports	0.980	0.980
technology	0.760	0.850

The biggest gains are in the two weakest classes, business (+8.5 points) and technology (+9.0), while sports and entertainment did not change. The baseline's main confusions, business → technology (9 articles) and politics → business (8), dropped to 1 and 3 with T5.

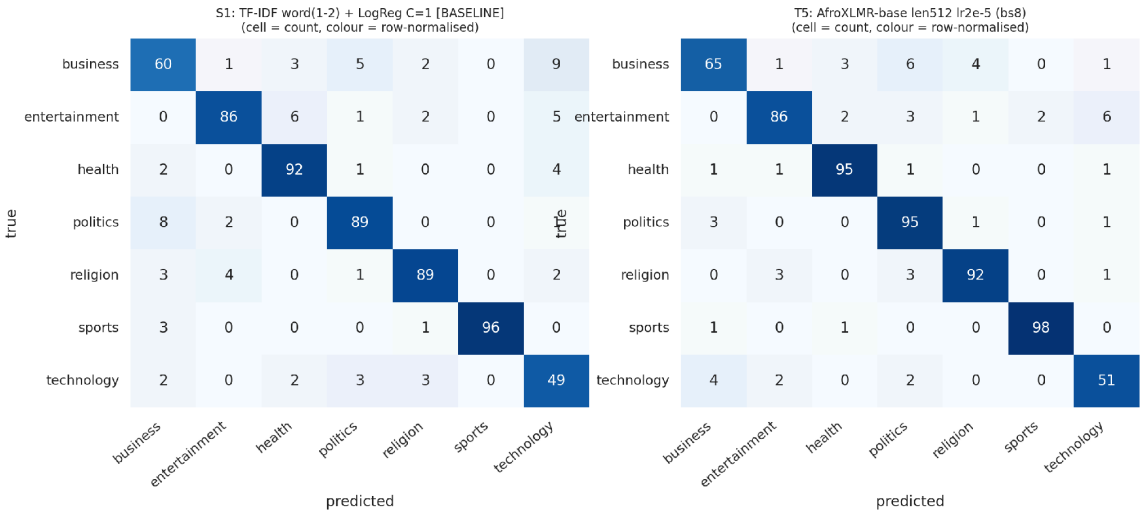


Figure 5. Test-set confusion matrices for the baseline (S1, left) and the selected model (T5, right). Cells show counts; colour shows the row-normalised share.

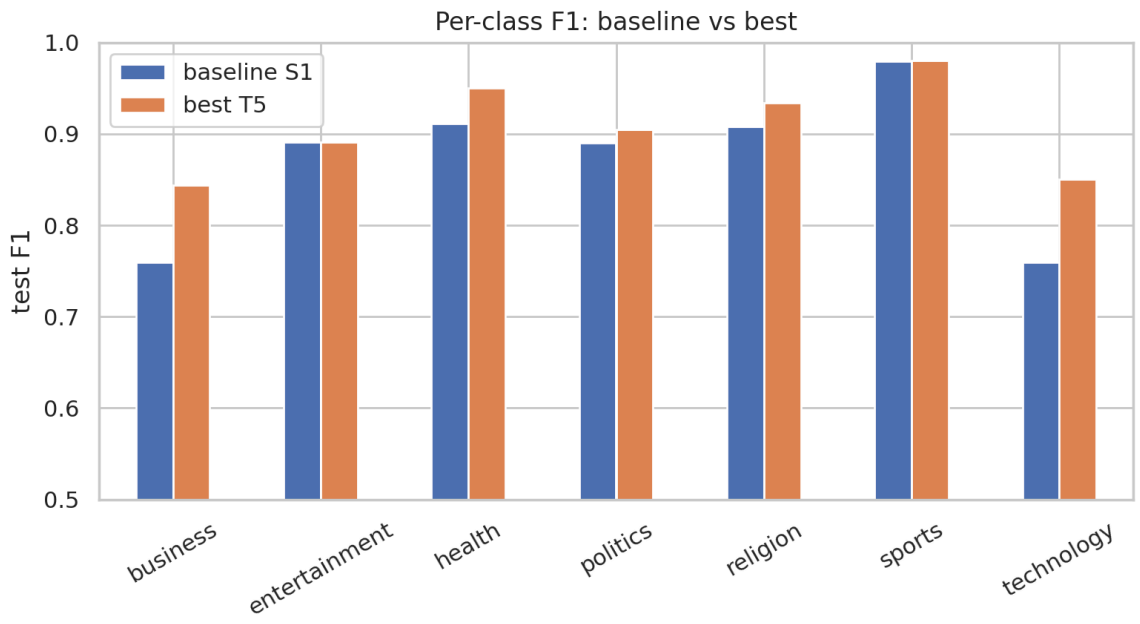


Figure 6. Per-class test F1 for the baseline (S1) and the selected model (T5).

7. Error Analysis and Discussion

The selected model T5 makes **55 errors out of 637** test articles (8.6%).

Where it succeeds. Sports is almost perfect (F1 0.98) because the vocabulary is very distinctive: match reports such as the Chelsea–Arsenal result and the Frank Ribery retirement story were classified with near-100% confidence. Health (0.95) and religion (0.93) are also strong. The baseline's own top features show why: sports is driven by *wasan, gasar, kofin, kwallon* (match, tournament, cup, football); health by *cutar, lafiya, coronavirus* (disease, health); politics by *jam'iyyar, zaben, apc* (party, election); business by *tattalin, naira, farashin, kudin* (economy, naira, prices, money).

Most common confusions (T5). business → politics (6), entertainment → technology (6), business → religion (4), technology → business (4).

Why it fails. Reading the misclassified articles (saved in `misclassified_test_examples.csv`), I see four main patterns:

1. **Genuine topic overlap.** An article on Saudi Arabia's financial losses from the pandemic-hit Hajj season is labelled business but is full of religious vocabulary; a road-safety story set around the Christmas season and a weekly news round-up that mentions Hajj pilgrims were also labelled business and predicted religion. Weekly "Waiwaye" round-ups cover many topics at once, so a single label is inherently unreliable.
2. **Debatable labels.** A story about Madonna catching COVID is labelled entertainment and predicted health; an article about Prince Philip's love of sport is labelled entertainment and predicted sports; a discovery of a planet outside the Milky Way is labelled entertainment and predicted technology. In each case the model's answer is defensible, so some "errors" are really label ambiguity.

3. **Cross-topic politics and religion.** A cleric's statement on Muslim–Muslim presidential tickets, labelled politics, was predicted religion, and an IMN/El-Rufa'i dispute, labelled religion, was predicted politics. Nigerian public life mixes the two.
4. **Short, caption-like articles and spelling noise.** Very short articles give little evidence. Spelling variation is real: my Word2Vec model's nearest neighbour for *daya* is *ɗaya* (the same word with and without the hooked letter), showing that the data contains both forms.

Overconfidence. All eight most confident mistakes had a predicted probability of about 0.996 or higher. The model's probabilities are therefore not calibrated, and a high score should not be read as proof of correctness.

Article length. Accuracy by length (S1 / T5): under 50 words 0.818 / 0.909; 50–150 words 0.911 / 0.911; 150–400 words 0.907 / 0.931; over 400 words 0.861 / 0.902. The Transformer helps most on the shortest and longest articles. The misclassified articles average 523 words, which fits the idea that long multi-topic stories are harder. They also had a slightly higher out-of-vocabulary rate (2.58% vs 1.88% for correct ones), a weak link to rare words and names.

Possible bias. The baseline's religious features are almost all Islamic terms (*musulunci, sheikh, malamin, musulmi*). This reflects the source data, and the model may be weaker on other religious content; I did not test this.

Link to the literature. The strong showing of Africa-focused pretraining agrees with Alabi et al. (2022) and Ogueji et al. (2021), who argue that models exposed to African languages during pretraining handle them better than generic multilingual models. Classical baselines were also part of the MasakhaNEWS evaluation (Adelani et al., 2023), and my results show why they remain worth including: a tuned TF-IDF model comes within about 2 points of the best Transformer.

8. Deployment

The selected model (T5, AfroXLMR-base fine-tuned with 512 tokens) is served with a **Gradio** web app (`app.py`). The user enters a Hausa headline and article text. The app joins them as "headline. text", tokenises with the model's tokeniser (truncating at 512 tokens), runs the fine-tuned classifier, applies softmax, and shows the probability of all seven topics, with the top class highlighted. The model is loaded from a local `best_model` folder or from the Hugging Face Hub. To run it locally: `pip install -r requirements.txt` (gradio, torch, transformers, sentencepiece), then `python app.py`.

Where it is hosted: It is hosted on HuggingFace and the deployment link is here. It can not last for long because the app was deployed on a free channel and can only hold it for up to 1 week maximum.

9. Limitations and Future Work

- **Single run per experiment.** Each configuration used one seed, so small differences (for example LSTM vs GRU, or T2 vs T5) are within noise. Repeating with several seeds and reporting mean ± standard deviation would fix this.
- **Small test set.** With 637 articles, one article moves accuracy by 0.16 points.

- **Narrow domain.** All data is professional news; performance on social media or informal Hausa is unknown.
- **Single-label, sometimes ambiguous labels.** Multi-label classification, or a re-annotation of the ambiguous cases, would suit the data better.
- **Possible religion bias.** See Section 7; this should be tested with non-Islamic religious articles.
- **Truncation.** Even at 512 tokens, about half of the training articles are cut. Chunking or hierarchical models could use the full text.
- **Uncalibrated confidence.** Calibration methods such as temperature scaling would make the displayed probabilities more trustworthy.
- **One uncontrolled comparison.** AfriBERTa and AfroXLMR differed in learning rate as well as in model.
- **More to try with more time or compute.** Larger African-language models, data augmentation, and tuning the sparse baseline as thoroughly as the Transformers.

10. Conclusion

On Hausa news topic classification, a fine-tuned AfroXLMR model improves significantly over a TF-IDF + Logistic Regression baseline (0.871 → 0.908 macro-F1), mostly by fixing the weakest classes, business and technology. The most important lessons were that pretraining on African languages and full fine-tuning matter far more than sequence length or learning rate, that Word2Vec embeddings are essential for RNNs on small data, and that simple, tuned sparse models remain strong competitors. The errors left are mostly articles that sit on the border between two topics, which points to the labels, not only the model, as the next thing to improve. The whole pipeline, from data analysis through experiments to a working web app, is reproducible from the repository.

11. References

Abid, A., Abdalla, A., Abid, A., Khan, D., Alfozan, A., & Zou, J. (2019). *Gradio: Hassle-free sharing and testing of ML models in the wild*. arXiv:1906.02569.

Adelani, D. I., Masiak, M., Azime, I. A., Alabi, J., Tonja, A. L., Mwase, C., et al. (2023). MasakhaNEWS: News topic classification for African languages. In *Proceedings of IJCNLP-AACL 2023*. arXiv:2304.09972.

Alabi, J. O., Adelani, D. I., Mosbach, M., & Klakow, D. (2022). Adapting pre-trained language models to African languages via multilingual adaptive fine-tuning. In *Proceedings of COLING 2022*.

Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. In *ICLR 2015*.

Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of EMNLP 2014*.

Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F., et al. (2020). Unsupervised cross-lingual representation learning at scale. In *Proceedings of ACL 2020*.

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT 2019*.

Hedderich, M. A., Adelani, D., Zhu, D., Alabi, J., Markus, U., & Klakow, D. (2020). Transfer learning and distant supervision for multilingual transformer models: A study on African languages. In *Proceedings of EMNLP 2020*.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735–1780.

Joshi, P., Santy, S., Budhiraja, A., Bali, K., & Choudhury, M. (2020). The state and fate of linguistic diversity and inclusion in the NLP world. In *Proceedings of ACL 2020*.

Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. In *ICLR 2019*.

McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika, 12*(2), 153–157.

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). *Efficient estimation of word representations in vector space*. arXiv:1301.3781.

Nekoto, W., Marivate, V., Matsila, T., Fasubaa, T., et al. (2020). Participatory research for low-resourced machine translation: A case study in African languages. In *Findings of EMNLP 2020*.

NLLB Team. (2022). *No language left behind: Scaling human-centered machine translation*. arXiv:2207.04672.

Ogueji, K., Zhu, Y., & Lin, J. (2021). Small data? No problem! Exploring the viability of pretrained multilingual language models for low-resourced languages. In *Proceedings of the 1st Workshop on Multilingual Representation Learning*.

Paszke, A., Gross, S., Massa, F., Lerer, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. In *NeurIPS 2019*.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Řehůřek, R., & Sojka, P. (2010). Software framework for topic modelling with large corpora. In *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks*.

Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management, 24*(5), 513–523.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In *NeurIPS 2017*.

Wolf, T., Debut, L., Sanh, V., Chaumond, J., et al. (2020). Transformers: State-of-the-art natural language processing. In *Proceedings of EMNLP 2020: System Demonstrations*.

Masakhane. *masakhane/masakhanews* [Dataset, AFL-3.0]. Hugging Face. https://huggingface.co/datasets/masakhane/masakhanews

Pretrained models: `Davlan/afro-xlmr-base`, `castorini/afriberta_base`, `xlm-roberta-base`. Hugging Face Hub.